

■ InsightScan

Automatisierte Belegprüfung mit Computer Vision, RAG und regelbasierter Compliance-Prüfung

Portfolio-Projekt im Bereich KI / Angewandte Informatik

InsightScan ist ein End-to-End-System, das Kassenbelege automatisch ausliest, gegen eine Spesenrichtlinie prüft und das Ergebnis verständlich zusammenfasst. Das Projekt kombiniert gezielt drei zentrale Bereiche moderner KI-Entwicklung: **Computer Vision** (Belegverständnis durch ein feingetunttes Layout-Modell), **Retrieval-Augmented Generation** (semantische Suche in Richtliniendokumenten) und **MLOps** (Monitoring der Modellqualität im laufenden Betrieb). Bewusst wurde außerdem eine **Hybrid-Architektur** gewählt: harte, eindeutige Geschäftsregeln laufen in deterministischem Code, ein lokales Sprachmodell übernimmt nur die verständliche Formulierung - eine Lektion, die aus einem konkret beobachteten Rechenfehler des Sprachmodells entstand (siehe Kapitel 'Herausforderungen & Lösungen').

Kategorie	Eckdaten
Datensatz	SROIE 2019 (626 annotierte Kassenbelege)
Modell-Genauigkeit	91 % F1-Score (Computer-Vision-Feldererkennung)
Baseline-Vergleich	Tesseract + Regex: ~20-40 % vs. LayoutLM: 85-100 % je Feld
Entwicklungsumgebung	MacBook Air M3, lokal, ohne Cloud-Kosten
Oberfläche	Streamlit-Web-App mit eigenem Corporate Design

1. Projektübersicht

Ausgangslage und Motivation

Ziel war der Aufbau eines KI-Portfolios, das die drei für den Arbeitsmarkt relevantesten KI-Teilbereiche abdeckt: Computer Vision, Generative AI / RAG und MLOps. Statt drei isolierte Übungsprojekte zu bauen (wie es in vielen Portfolios üblich ist), wurden alle drei Bereiche bewusst in **einem zusammenhängenden Produkt** vereint - das erzeugt eine kohärente Geschichte statt dreier Fingerübungen und zeigt, wie die Bereiche in der Praxis ineinandergreifen.

Die Idee: InsightScan

InsightScan liest Kassenbelege (fotografiert oder gescannt) automatisch aus, prüft die erkannten Werte (Firma, Datum, Betrag) gegen eine hinterlegte Spesenrichtlinie und erklärt das Ergebnis in verständlicher Sprache - inklusive Angabe, welche Freigabestufe ggf. nötig ist. Das ist ein realistischer Anwendungsfall aus dem Finance-/Controlling-Bereich, der auch außerhalb reiner Tech-Firmen überzeugt.

Die drei Säulen im Überblick

- **Computer Vision:** Ein LayoutLM-Modell (Layout- und textbasierte Dokumentenanalyse) wird per Fine-Tuning darauf trainiert, Firma, Datum und Gesamtbetrag auf Belegbildern zu erkennen.
- **RAG (Retrieval-Augmented Generation):** Eine Spesenrichtlinie wird in durchsuchbare Abschnitte zerlegt; per semantischer Ähnlichkeitssuche werden die relevantesten Passagen zu einem konkreten Beleg gefunden und einem lokalen Sprachmodell als Kontext mitgegeben.
- **MLOps:** Ein Monitoring-Dashboard protokolliert bei jeder Verarbeitung, ob die gelernte Modell-Erkennung erfolgreich war oder ein Regel-Fallback einspringen musste - ein konkreter, messbarer Indikator für 'Data Drift', wenn neue Beleg-Formate häufiger auftreten.

Bewusste Architektur-Entscheidung: Hybrid statt Alles-LLM

Ein früher Test zeigte, dass das lokale Sprachmodell (gemma3) bei einfachen Zahlenvergleichen (z. B. 'liegt 193 unter 100?') sachlich falsche Antworten gab - eine bekannte Schwäche kleinerer Sprachmodelle. Als Konsequenz wurde die Architektur bewusst zweigeteilt: Alle harten, eindeutigen Geschäftsregeln (Betragsgrenzen, Fristen) laufen in reinem, deterministischem Python-Code; das Sprachmodell bekommt das bereits korrekt berechnete Ergebnis und übernimmt ausschließlich die verständliche, freundliche Formulierung. Diese Entscheidung ist eines der zentralen technischen Learnings dieses Projekts.

2. Eingesetzte Skills & Technologien

Programmiersprache & Umgebung

Bereich	Werkzeug / Technologie
Sprache	Python 3.13
Entwicklungsumgebung	Visual Studio Code, integriertes Terminal
Versions-/Paketverwaltung	venv (virtuelle Umgebung), pip
Hardware	Apple MacBook Air M3 (Apple Silicon, MPS-Backend für PyTorch)
Betriebssystem-Werkzeuge	Homebrew (Tesseract-Installation), Bash/Zsh

Computer Vision / Machine Learning

Bibliothek	Einsatzzweck
PyTorch	Deep-Learning-Framework, Modelltraining und Inferenz
Hugging Face Transformers	LayoutLM-Modellarchitektur, Tokenisierung, Trainer-API
segeval	Precision / Recall / F1-Berechnung für Token-Klassifikation
Tesseract OCR (pytesseract)	Texterkennung und Wortpositionserkennung auf Belegbildern
Pillow (PIL)	Bildverarbeitung

RAG / Generative KI

Bibliothek / Tool	Einsatzzweck
sentence-transformers	Erzeugen semantischer Embeddings (all-MiniLM-L6-v2)
NumPy	Kosinus-Ähnlichkeitsberechnung für die semantische Suche
Ollama + gemma3	Lokal laufendes Sprachmodell zur Textgenerierung (kostenlos, offline)
Prompt Engineering	Gezielte Prompt-Gestaltung, um Halluzinationen bei Zahlen zu vermeiden

MLOps & Anwendungsentwicklung

Werkzeug	Einsatzzweck
Streamlit	Interaktive Web-Oberfläche mit eigenem Corporate Design (HTML/CSS)
JSON Lines (JSONL)	Datensparsames Protokollformat für das Monitoring
Eigenes Monitoring-Modul	Fallback-Quote, Trend-Analyse als Data-Drift-Indikator
Kaggle API	Download des SROIE-2019-Trainingsdatensatzes

Übergreifende Fähigkeiten

- Systematisches, evidenzbasiertes Debugging über mehrere Ebenen hinweg (Daten → Modell → OCR-Konfiguration)
- Kritische Bewertung von KI-Ausgaben (Erkennen einer LLM-Fehlberechnung statt blindem Vertrauen)
- Bewusste Sicherheitsentscheidungen (z. B. `weights_only=True` beim Laden von Modellgewichten, Vermeidung von Pickle-Risiken)
- Datenschutzbewusstes Design (Monitoring speichert keine Beleginhalte, nur Erfolgsindikatoren)
- Dokumentation von Grenzen und offenen Punkten statt Übertreibung der Ergebnisse

3. Architektur & Datenfluss

Der komplette Verarbeitungsweg von einem hochgeladenen Belegfoto bis zur fertigen Compliance-Zusammenfassung gliedert sich in sechs Schritte:

- **1. OCR-Erkennung** (Tesseract, PSM 6): Wörter und ihre Positionen werden aus dem Bild extrahiert und zeilenweise gruppiert.
- **2. Computer-Vision-Klassifikation** (LayoutLM): Jedes Wort wird anhand von Text *und* Position als Firma, Datum, Betrag oder irrelevant eingestuft.
- **3. Regex-Fallback:** Liefert LayoutLM kein sinnvolles Ergebnis (z. B. bei stark abweichenden Belegformaten), greift eine einfachere, musterbasierte Ersatzerkennung.
- **4. Deterministische Regelprüfung:** Betragsgrenzen und Fristen werden in reinem Python-Code geprüft - unabhängig von der gewählten Währung.
- **5. RAG-Suche:** Die relevantesten Abschnitte der Spesenrichtlinie werden per semantischer Ähnlichkeit gefunden.
- **6. Sprachmodell-Zusammenfassung:** Das bereits feststehende Prüfergebnis wird von gemma3 in eine freundliche, verständliche Nachricht übersetzt.

Wichtiges Architekturprinzip: Jede Stufe ist unabhängig testbar und austauschbar. So ließ sich z. B. die OCR-Komponente (Tesseract) komplett gegen die perfekten SROIE-Annotationen austauschen, ohne den Rest der Pipeline zu verändern - ein Beleg für sauber getrennte Verantwortlichkeiten.

4. Computer Vision: Datenaufbereitung & Modelltraining

4.1 Datensatz

Als Trainingsgrundlage dient der öffentliche **SROIE-2019-Datensatz** (Scanned Receipts OCR and Information Extraction): 626 annotierte Kassenbelege mit Ground-Truth-Werten für Firma, Datum, Adresse und Gesamtbetrag, jeweils inklusive Wortpositionen (Bounding Boxes).

4.2 Automatisches Erzeugen der Trainingslabels

Die reinen Ground-Truth-Werte mussten zunächst mit den passenden Wörtern/Positionen im Beleg abgeglichen werden (BIO-Tagging). Eine besondere Herausforderung war die Zuordnung des Gesamtbetrags: Label ('TOTAL:') und Betrag standen im Rohtext oft nicht in derselben Zeile, sondern lediglich auf gleicher visueller Höhe. Die Lösung kombiniert Positionsvergleich und einen einfacheren Fallback:

```
def find_total_position_based(line_entries, normalized_total):
    """Sucht den Betrag auf ähnlicher Höhe wie eine 'total'-Zeile."""
    total_label_entries = [e for e in line_entries if "total" in normalize_text(e["text"])]
    for total_entry in total_label_entries:
        target_y = y_center(total_entry["bbox"])
        for entry in line_entries:
            if abs(y_center(entry["bbox"]) - target_y) > ROW_Y_TOLERANCE:
                continue
            for i, word in enumerate(entry["words"]):
                if normalize_amount(word) == normalized_total:
                    entry["labels"][i] = "B-TOTAL"
                    return True
    return False

def find_total_fallback(line_entries, normalized_total):
    """Fallback: den gesamten Beleg nach dem exakten Betrag durchsuchen."""
    for entry in line_entries:
        for i, word in enumerate(entry["words"]):
            if normalize_amount(word) == normalized_total:
                entry["labels"][i] = "B-TOTAL"
                return True
    return False
```

Ausschnitt aus build_layoutlm_dataset.py - zweistufige Total-Erkennung (Position, dann Fallback)

4.3 Fine-Tuning des LayoutLM-Modells

Das mitgelieferte, vortrainierte layout lm-base-uncased-Modell stammt aus dem ursprünglichen Microsoft-Forschungscode und speichert seine Gewichte unter dem internen Namen bert . * statt des von der Transformers-Bibliothek erwarteten layout lm . *. Ohne Korrektur würden dadurch **alle** vortrainierten Gewichte verworfen und durch zufällige Werte ersetzt - ein Fehler, der sich erst durch Analyse des Trainingsverlaufs (sinkender Trainings-Loss bei stagnierender Validierungsgüte) aufdeckte:

```
def load_pretrained_layoutlm(config):
    model = LayoutLMForTokenClassification(config)
    weights_path = BASE_MODEL_DIR / "pytorch_model.bin"
    # weights_only=True: Sicherheitsmaßnahme gegen potenziell schädlichen Code
    raw_state_dict = torch.load(weights_path, map_location="cpu", weights_only=True)

    remapped_state_dict = {}
    for key, value in raw_state_dict.items():
        if key.startswith("bert."):
            new_key = "layoutlm." + key[len("bert."):]
            remapped_state_dict[new_key] = value
        elif key.startswith("cls."):
            continue # alter Masked-Language-Model-Kopf wird nicht gebraucht
        else:
            remapped_state_dict[key] = value

    model.load_state_dict(remapped_state_dict, strict=False)
    return model
```

Ausschnitt aus train_layoutlm.py - manuelle Gewichts-Umbenennung als Fehlerkorrektur

4.4 Trainingsergebnis

Nach der Korrektur verbesserte sich der F1-Score drastisch:

Feld	Vorher (fehlerhafte Gewichte)	Nachher (korrigiert)
COMPANY	34 %	85 %
DATE	54 %	100 %

TOTAL	20 %	85 %
Gesamt (F1)	39 %	91 %

5. Baseline-Vergleich: Tesseract+Regex vs. LayoutLM

Vor dem Fine-Tuning wurde bewusst eine einfachere Baseline gebaut (Tesseract-OCR + Regex-Feldextraktion), um den Mehrwert des Machine-Learning-Ansatzes messbar zu belegen, statt ihn nur zu behaupten:

Feld	Baseline (Tesseract + Regex)	LayoutLM (Fine-Tuned)
Firma	20,9 % (strenge exakte Übereinstimmung)	85 % F1
Datum	39,1 %	100 % F1
Betrag	31,9 %	85 % F1

Die Baseline-Auswertung deckte zusätzlich typische, nachvollziehbare Fehlerquellen auf: zu enge Regex-Muster (z. B. nur 4-stellige Jahreszahlen), Mehrfachtreffer bei identischen Beträgen innerhalb eines Belegs sowie klassische OCR-Verwechslungen (Komma statt Punkt, '3' statt '?').

6. Hybrid-Architektur: Regeln + Sprachmodell

Der ausschlaggebende Fund

Bei einem frühen Test wurde gemma3 gebeten, direkt zu beurteilen, ob ein Betrag von 193,00 unter einer Grenze von 100,00 liegt. Die Antwort des Modells war eindeutig falsch ('liegt unter der Grenze von 100,00'). Dieser Fund führte zur zentralen Architektur-Entscheidung, exakte Zahlenvergleiche niemals dem Sprachmodell zu überlassen.

Die Lösung: Regeln in Code, Formulierung im LLM

```
def evaluate_amount_rule(total_str, currency="RM"):
    amount = float(str(total_str).replace(",", "."))

    if amount <= 100.00:
        return {"requires_approval": False, "approval_level": "keine", ...}
    elif amount <= 500.00:
        return {"requires_approval": True,
                "approval_level": "direkte Führungskraft", ...}
    elif amount <= 2000.00:
        return {"requires_approval": True,
                "approval_level": "Abteilungsleitung", ...}
    else:
        return {"requires_approval": True,
                "approval_level": "vorherige schriftliche Genehmigung", ...}
```

Ausschnitt aus compliance_rules.py - deterministische Betragsprüfung, währungsunabhängig

Der Prompt an das Sprachmodell enthält das **bereits berechnete** Ergebnis mit der expliziten Anweisung, es nicht neu zu berechnen, sondern nur verständlich zu formulieren - dadurch bleibt die Ausgabe faktisch korrekt, während das Sprachmodell seine eigentliche Stärke (natürliche Sprache) ausspielt.

Universelle Währungsunterstützung

Die Geschäftslogik vergleicht ausschließlich reine Zahlenwerte; die Währung wird als frei eintippbarer Parameter durch die gesamte Pipeline gereicht. Eine neue Währung erfordert keine Code-Änderung.

7. RAG: Semantische Richtliniensuche

Die Spesenrichtlinie wird anhand ihrer Markdown-Überschriften in inhaltlich abgeschlossene Abschnitte zerlegt und mit einem kompakten Embedding-Modell (all-MiniLM-L6-v2, ca. 80 MB, vollständig lokal) in numerische Bedeutungsvektoren umgewandelt. Zur Laufzeit wird der passendste Abschnitt per Kosinus-Ähnlichkeit gefunden:

```
def find_relevant_chunks(query, policy_index, embedding_model, top_k=3):
    query_embedding = embedding_model.encode(query)
    scored_chunks = []
    for entry in policy_index:
        chunk_embedding = np.array(entry["embedding"])
        similarity = cosine_similarity(query_embedding, chunk_embedding)
        scored_chunks.append((similarity, entry["text"]))
    scored_chunks.sort(key=lambda pair: pair[0], reverse=True)
    return scored_chunks[:top_k]
```

Ausschnitt aus pipeline.py - Retrieval-Schritt des RAG-Systems

8. Herausforderungen & Lösungen

Die folgenden Debugging-Episoden waren besonders lehrreich und zeigen einen systematischen, evidenzbasierten Umgang mit unklaren Fehlern - jede Vermutung wurde an echten Daten überprüft, bevor eine Lösung umgesetzt wurde.

8.1 Fehlende Zahlen bei der OCR-Erkennung

Symptom: Auf echten (nicht-SROIE-)Belegen lieferte Tesseract weder Firma noch Betrag. **Diagnose:** Tesseracts automatische Seitensegmentierung übersah komplette Tabellenspalten mit isolierten Zahlen. **Lösung:** Umstellung auf den Modus - psm 6 (gesamte Seite als ein zusammenhängender Textblock).

8.2 Firmenname trotz erkannter Wörter nicht klassifiziert

Symptom: Nach Behebung von 8.1 wurden alle Wörter korrekt erkannt, LayoutLM stufte den Firmennamen aber weiterhin als irrelevant ein. **Diagnose:** Die Trainingsdaten enthielten breite, zeilenweite Positionsboxen; Tesseract lieferte dagegen enge Einzelwort-Boxen - eine strukturelle Diskrepanz zwischen Training und Anwendung (Distribution Shift). **Lösung:** OCR-Wörter werden vor der Modellklassifikation zu Zeilen gruppiert und erhalten eine gemeinsame Positionsbox, die die Trainingsdaten-Struktur nachbildet.

8.3 Generalisierungsgrenze bei fremdsprachigen/andersformatigen Belegen

Test: Ein selbst erstellter, deutschsprachiger Kassenbon ('Der Saftladen') mit Punkt-Datumsformat und Euro-Beträgen wurde bewusst als Grenztest hochgeladen. **Ergebnis:** Firma und Datum wurden von LayoutLM nicht bzw. falsch erkannt - das Modell wurde ausschließlich auf englischsprachigen, malaysischen Belegen trainiert und generalisiert entsprechend schlecht auf strukturell und sprachlich abweichende Formate. **Lösung:** Ergänzung eines Regex-basierten Fallbacks für Datum und Betrag, der bei ungültigem oder fehlendem Modell-Ergebnis einspringt.

```
date_is_valid = date and parse_receipt_date(date) is not None
if not date_is_valid:
    fallback_date = extract_date_fallback(raw_text)
    if fallback_date and parse_receipt_date(fallback_date) is not None:
        date = fallback_date
        used_fallback["date"] = True
```

Ausschnitt aus pipeline.py - Fallback greift auch bei inhaltlich ungültigem Modell-Ergebnis

Bewusst nicht behoben: Ein Nachtraining mit zusätzlichen, selbst annotierten deutschsprachigen Belegen wurde erwogen, aber verworfen - der Aufwand (Sammeln, manuelles Annotieren mit Bounding-Boxes) stünde in keinem Verhältnis zum Nutzen für dieses Portfolio-Projekt. Diese Grenze wird stattdessen bewusst offen dokumentiert.

9. MLOps: Monitoring-Dashboard

Jede Beleg-Verarbeitung protokolliert automatisch und datensparsam (nur Erfolgs-/Fallback-Indikatoren, keine Beleginhalte), ob die gelernte Modell-Erkennung erfolgreich war:

```
def compute_stats(logs):
    total = len(logs)
    company_recognized_count = sum(1 for e in logs if e["company_recognized"])
    date_fallback_count = sum(1 for e in logs if e["date_used_fallback"])
    total_fallback_count = sum(1 for e in logs if e["total_used_fallback"])
    return {
        "total": total,
        "company_recognition_rate": company_recognized_count / total,
        "date_fallback_rate": date_fallback_count / total,
        "total_fallback_rate": total_fallback_count / total,
    }
}
```

Ausschnitt aus monitoring.py - zentrale Kennzahlenberechnung

Die Fallback-Quote dient als konkreter **Data-Drift-Indikator**: Steigt sie über Zeit an, weichen die eingereichten Belege zunehmend von den Trainingsdaten ab - ein objektives Signal für ein mögliches Nachtraining, statt auf ein reines Bauchgefühl angewiesen zu sein.

Bisher verarbeitete Belege: 16

Firma erkannt

56%

Datum: Fallback-Quote

62%

Betrag: Fallback-Quote

62%

Die Fallback-Quote zeigt, wie oft die einfache Regex-Ersatzerkennung statt des trainierten LayoutLM-Modells einspringen musste. Eine dauerhaft steigende Quote deutet auf 'Data Drift' hin - die eingehenden Belege weichen zunehmend von den Trainingsdaten ab, was ein Signal für ein Nachtraining mit neueren, vielfältigeren Beispielen wäre.

Trend: Betrag-Fallback-Quote über Zeit ⇄



Screenshot des Monitoring-Dashboards nach 16 verarbeiteten Belegen: Firma-Erkennungsrate 56 %, Datum- und Betrag-Fallback-Quote je 62 %. Der sichtbare Anstieg im Trend-Diagramm resultiert aus den bewusst gemischten Testbelegen (SROIE-Belege und der abweichend formatierte deutsche Beleg).

Ehrliche Einordnung: Mit 16 verarbeiteten Belegen demonstriert das Dashboard das Konzept nachvollziehbar, ist aber statistisch noch nicht aussagekräftig. Für eine belastbare Trendanalyse wären deutlich mehr verarbeitete Belege nötig, als im Rahmen dieses Projekts gesammelt werden konnten.

10. Anwendung: Streamlit-Oberfläche

Die Web-Oberfläche bietet zwei Wege, einen Beleg einzureichen (eigener Upload oder Beispielbeleg), eine frei editierbare Währungsangabe, klar strukturierte Ergebnisdarstellung mit Ampel-Farben (grün/gelb/rot je nach Prüfergebnis), einen ausklappbaren Transparenzbereich für die RAG-Nachvollziehbarkeit sowie das Monitoring-Dashboard als dritten Tab. Das Erscheinungsbild (gelbes Farbschema mit dezentem Währungssymbol-Wasserzeichen) wurde vollständig über eigenes CSS umgesetzt.

Caching-Strategie

Da das Laden aller Modelle (LayoutLM, Embedding-Modell) mehrere Sekunden dauert, wird `@st.cache_resource` genutzt, damit dies nur einmalig beim App-Start geschieht, nicht bei jeder Nutzerinteraktion.

11. Grenzen & Ausblick

- **Sprachliche/formatbedingte Generalisierung:** Das Modell ist auf englischsprachige, malaysische Belege spezialisiert; andere Sprachen/Formate benötigen den Regex-Fallback oder ein erweitertes Training.
- **Adressfeld nicht implementiert:** Die Erkennungsrate für mehrzeilige Adressen lag bei nur 1,9 % (Ground-Truth-Adressen erstrecken sich über mehrere Textzeilen, unser Matching verglich aber nur einzelne Zeilen) - das Feld wurde bewusst aus dem Projektumfang entfernt statt unzuverlässig einzubauen.
- **Monitoring-Stichprobe:** Aktuell demonstriert das Dashboard das Konzept, bräuchte für belastbare Aussagen aber deutlich mehr verarbeitete Belege.
- **Denkbare Erweiterung:** Automatische Währungserkennung direkt aus dem Belegbild statt manueller Eingabe.
- **Denkbare Erweiterung:** Nachtraining mit einem vielfältigeren, mehrsprachigen Datensatz zur Verbesserung der Generalisierung.
- **Denkbare Erweiterung:** Automatisierter Retraining-Trigger, sobald die Fallback-Quote einen definierten Schwellenwert überschreitet (echtes MLOps-Continuous-Training).

12. Setup & Ausführung

```
# Projekt-Setup
cd insightscan
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt # torch, transformers, streamlit, ...

# Voraussetzung: Ollama mit gemma3 lokal installiert und gestartet
ollama pull gemma3

# Anwendung starten
streamlit run app/streamlit_app.py
```

Kurzanleitung zum lokalen Ausführen des Projekts

*InsightScan - Portfolio-Projekt | Computer Vision (LayoutLM) · RAG (sentence-transformers + Ollama/gemma3) ·
MLOps (Fallback-Monitoring) · Hybrid-KI-Architektur*